Région Hauts-de-France

Cofinancé par l'Union européenne

realconseil
DIAGNOSTIC ACCOMPAGNEMENT FORMATION

SYMFONY 6 – Le Register et la sécurité

Introduction

Le login est réalisé il nous faut maintenant avant de nous connecter notre formulaire d'inscription.

Nous allons voir comment créer l'inscription d'un user avec les sécurités de symfony6 par la même occasion

I) Création du form

Petit rappel pour faire la création du formulaire Register nous allons faire la commande « php bin/console make:form » et l'appeler RegistrationType comme ci-dessous

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   GITLENS

PS C:\Users\alexa\OneDrive\Bureau\workspace\symfonytuto\firstproject> php bin/console make:form

 The name of the form class (e.g. VictoriousPizzaType):
 > RegistrationType
```

On lui dit que c'est basé sur l'entity User

```
 The name of Entity or fully qualified model class name that the new form will be bound to (empty for
 > User
```

Jusque là rien de surprenant vous avez déjà fait des choses similaires

Regardons ce qu'il nous a créé :

```
5   use App\Entity\User;
6   use Symfony\Component\Form\AbstractType;
7   use Symfony\Component\Form\FormBuilderInterface;
8   use Symfony\Component\OptionsResolver\OptionsResolver;
9
10  class RegistrationType extends AbstractType
11  {
12      public function buildForm(FormBuilderInterface $builder, array $options): void
13      {
14          $builder
15              ->add('email')
16              ->add('roles')
17              ->add('password')
18              ->add('name')
19              ->add('pseudo')
20              ->add('createdAt')
21          ;
22      }
23
24      public function configureOptions(OptionsResolver $resolver): void
25      {
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   GITLENS
```

Nous avons ici toutes les propriétés de notre user nous allons devoir tout paramétrer pour tout sécuriser au niveau des constraints comme nous l'avons fait pour les formulaires ingredient et recipe

Nous allons d'abord supprimer le rôle et le createdAt nous n'en avons pas besoin, par contre il nous manque le submit

Voici le code modifié :

```php
use Symfony\Component\Form\FormBuilderInterface;
use Symfony\Component\OptionsResolver\OptionsResolver;

class RegistrationType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('email')
            ->add('password')
            ->add('name')
            ->add('pseudo')
            ->add('submit')
        ;
    }

    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class' => User::class,
```

Modifions notre builder en renseignant le type pour chaque propriété avec les class, les constraints etc….

```php
$builder
    ->add('email',EmailType::class,[
        'attr'=>[
            'class'=>'form-control',
            'minLenght'=>'2',
            'maxLenght'=>'180'
        ],
        'label'=>'Adress email',
        'label_attr'=>[
            'class'=>'form-label'
        ],
        'constraints'=>[
            new Assert\NotBlank(),
            new Assert\Email(),
            new Assert\Length(['min'=> 2,'max'=> 180])
        ]

    ])
```

```php
    ->add('name',TextType::class,[
        'attr'=>[
            'class'=>'form-control',
            'minLenght'=>'2',
            'maxLenght'=>'50'
        ],
        'label'=>'Nom',
        'label_attr'=>[
            'class'=>'form-label'
        ],
        'constraints'=>[
            new Assert\NotBlank(),
            new Assert\Length(['min'=> 2,'max'=> 50])
        ]
    ])
```

```php
    ->add('pseudo',TextType::class,[
        'attr'=>[
            'class'=>'form-control',
            'minLenght'=>'2',
            'maxLenght'=>'50'
        ],
        'label'=>'Pseudo',
        'label_attr'=>[
            'class'=>'form-label'
        ],
        'constraints'=>[
            new Assert\Length(['min'=> 2,'max'=> 50])
        ]
    ])
    ->add('submit',SubmitType::class,[
        'attr'=>[
            'class'=>'btn btn-primary'
        ]
    ])
;
```

En ce qui concerne le password nous allons regarder le repeatedType. Car nous voulons une confirmation du mot de passe lors de l'inscription

Prenons donc la documentation du RepeatedType , nous allons chercher second_options car nous allons dire que nous voulons la même chose que le premier champ

```
38            ->add('plainPassword',RepeatedType::class,[
39                'type'=>PasswordType::class,
40                'first_options'=>[
41                    'label'=>'Mot de passe',
42                ],
43                'second_options'=>[
44                    'label'=>"Confirmation du mot de passe"
45                ],
46                'invalid_message'=>"Les mots de passe ne correspondent pas"
47            ])
```

II) Le controller

Nous allons utiliser le loginController pour faire le controller du register

```
    {
        // controller can be blank: it will never be called!
        throw new Exception('Don\'t forget to activate logout in security.yaml');
    }

    #[Route('/inscription', name: 'app_registration', methods:['GET','POST'])]
    public function registration():Response
    {
        return $this->render('pages/login/registration.html.twig');
    }
```

Nous reviendrons sur cette méthod par la suite afin de finir toutes les fonctionnalités requises

III) La view

Nous allons créer le fichier registration.html.twig dans le dossier login et faire notre formulaire

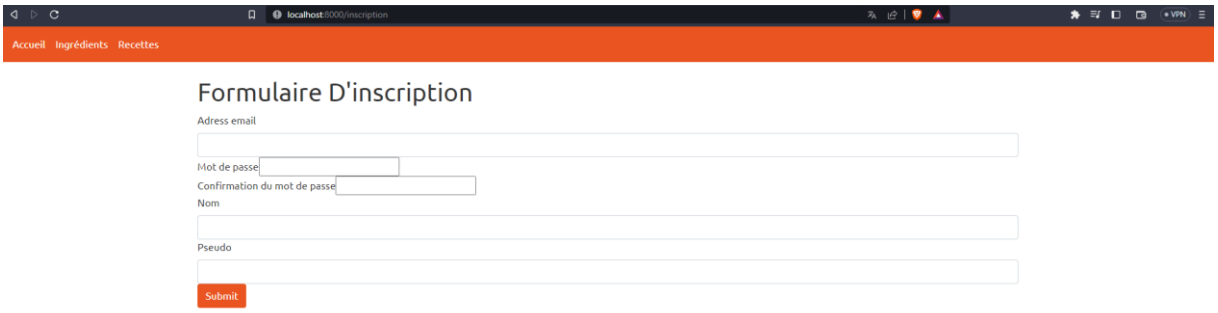
```
1   {% extends 'base.html.twig' %}
2
3   {% block title %}S'enregister{% endblock %}
4
5   {% block body %}
6       <div class="container">
7           <h1 class="mt-4">Formulaire D'inscription</h1>
8           {{ form(form) }}
9       </div
10  {% endblock %}
```

Pour le moment nous auons ceci et nous retournons dans le controller loginController pour faire le nécessaire en ce qui concerne l'affichage du formulaire comme ceci :

```php
#[Route('/inscription', name: 'app_registration', methods:['GET','POST'])]
public function registration():Response
{
    $user = new User();
    $form = $this->createForm(RegistrationType::class,$user);

    return $this->render('pages/login/registration.html.twig',[
        'form' => $form->createView()
    ]);
}
```

Niveau visuel nous avons notre formulaire dans le navigateur (pas très beau et nous modifierons cela par la suite)



Ajoutons les class dans notre registerType au niveau du plainpassword comme ceci

```php
->add('plainPassword',RepeatedType::class,[
    'type'=>PasswordType::class,
    'first_options'=>[
        'attr'=>[
            'class'=>'form-control'
        ],
        'label'=>'Mot de passe',
        'label_attr'=>[
            'class'=>'form-label'
        ],
    ],
    'second_options'=>[
        'attr'=>[
            'class'=>'form-control'
        ],
        'label'=>"Confirmation du mot de passe",
        'label_attr'=>[
            'class'=>'form-label'
        ],
    ],
    'invalid_message'=>"Les mots de passe ne correspondent pas"
])
```

On y ajoute le form-control et le form-label afin d'avoir le même visuel que les autres champs du formulaire

Mettons un label sur notre bouton, toujours dans notre registerType au niveau du submit :

```
85                'constraints'=>[
86                    new Assert\Length(['min'=> 2,'max'=> 50])
87                ]
88            ])
89            ->add('submit',SubmitType::class,[
90                'attr'=>[
91                    'class'=>'btn btn-primary mt-4'
92                ],
93                'label'=>'Enregistrer'
94            ])
95        ;
```

Le rendu sur le navigateur aura changer et le visuel du formulaire sera plus élégant et l'expérience utilisateur sera au rendez-vous :

Si maintenant nous souhaitons nous inscrire cela ne fonctionnera pas car nous n'avons pas renseigné notre controller à ce sujet.

IV) La logique du register

Retournons donc dans notre controller LoginController pour voir la logique du Register.

Nous allons devoir faire étape par étapes certaines choses :

- Faire le handleRequest du form
- Verif la soummission et la validation du formulaire
    - Si oui on récupère les données du formulaire ->getData()
        - On fait le persist du user
        - Et on flush
        - On fait la redirection vers le login
        - Un message flash success

```php
#[Route('/inscription', name: 'app_registration', methods:['GET','POST'])]
public function registration(Request $request, EntityManagerInterface $manager):Response
{
    $user = new User();
    $form = $this->createForm(RegistrationType::class,$user);

    $form->handleRequest($request);
    if($form->isSubmitted() && $form->isValid()){
        $user=$form->getData();

        $this->addFlash(
            'success',
            'Votre compte a bien été créé.'
        );      You, 2 seconds ago • Uncommitted changes

        $manager->persist($user);
        $manager->flush();

        return $this->redirectToRoute('app_login');
    }
    return $this->render('pages/login/registration.html.twig',[
        'form' => $form->createView()
    ]);
}
```

Nous allons retourner un bref instant dans notre view du Login pour faire l'affichage du message flash, car notre redirection se fait sur cette view et faisons cet affichage comme nous l'avons fait précédemment

```twig
{% block title %}Formulaire de Connexion{% endblock %}

{% block body %}
    <div class="container">
        <h2 class="mt-4">Se Connecter</h2>
        {% for message in app.flashes('success') %}
            <div class="alert alert-success mt-4">
                {{ message }}
            </div>
        {% endfor %}
        {% if error %}
        <div class="alert alert-dismissible alert-danger">
            {{ error.messageKey|trans(error.messageData, 'security') }}
        </div>

        {% endif %}      You, yesterday • loginForm ok

        <form action="{{ path('app_login') }}" method="post" class="mt-4">
```

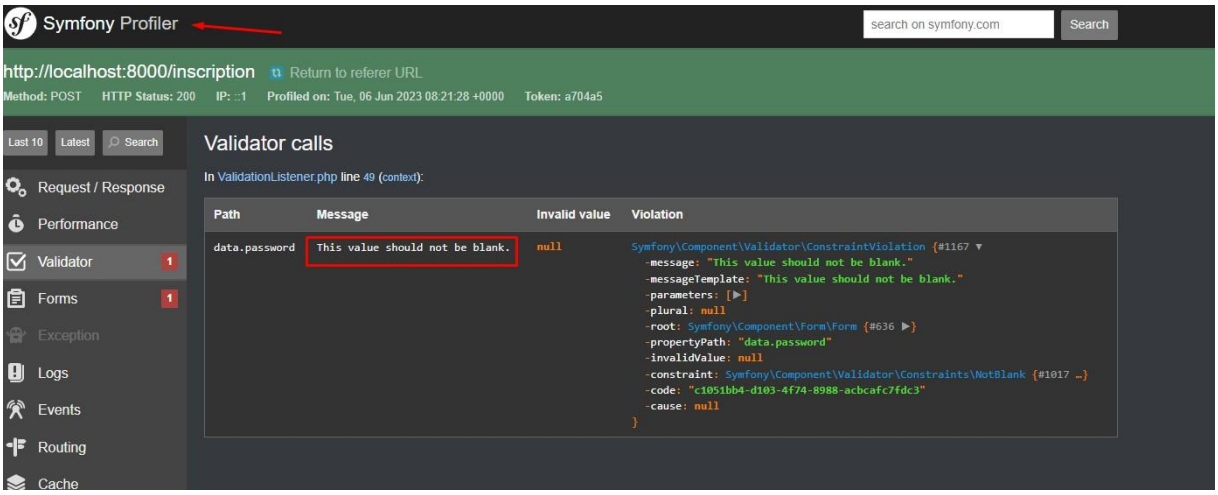
Nous allons donc maintenant essayer d'ajouter un user

Quand on clique sur le bouton un message d'erreur apparait

Nous allons débuguer cela ensemble

V) Debugage

Nous allons voir ensemble comment faire pour que notre user arrive enfin à s'inscrire.



Quand nous allons dans notre profiler nous avons ce retour d'erreur

Nous allons retourner dans notre controller et faire un dd($form->getData pour voir ce que notre formulaire nous retourne) à différents endroits dans notre code

Et dans le navigateur nous avons :

```
LoginController.php on line 45:
App\Entity\User {#639 ▼
  -id: null
  -email: "albert@gmail.com"
  -roles: []
  -plainPassword: "password"
  -password: null
  -name: "albert"
  -pseudo: "alberto"
  -createdAt: DateTimeImmutable @1686040499 {#640 ▼
    date: 2023-06-06 08:34:59.615842 UTC (+00:00)
  }
}
```

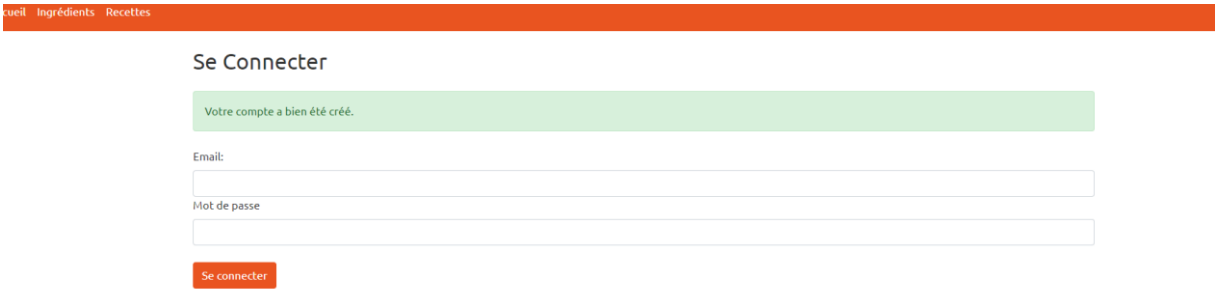
Nous voyons ici que password est à null

Nous allons devoir y mettre une valeur de base dans l'entity User au niveau de password comme ci-dessous

```
31      private ?string $plainPassword = null;
32
33      /**
34       * @var string The hashed password
35       */
36      #[ORM\Column]
37      #[Assert\NotBlank()]
38      private ?string $password = 'password';        You, 2 minut
39
40      #[Assert\NotBlank()]
41      #[Assert\Length(min:2,max:50)]
42      #[ORM\Column(length: 50)]
```

Ensuite nous allons enlever notre dd($form->getData()) de notre controller sinon notre user ne s'ajoutera jamais

Puis nous aurons ceci au niveau du visuel



Regardons dans la BDD, si notre user s'ajoute en bdd

Éditer	Copier	Supprimer	45	pfernandes@hotmail.fr	["ROLE_USER"]	$2y$13$DHZkzFuxo5KK1KdJxqYiQOZgjhbSL7rIYp2m1wTq96G...	Alex Delannoy-Gomez	NULL	2023-06-04 19:11:47
Éditer	Copier	Supprimer	46	henri.mary@tele2.fr	["ROLE_USER"]	$2y$13$UjmFIbukL9RueT8bubBnnO3fucPNYwRbQr1gFoTjtoG...	Yves Potier	NULL	2023-06-04 19:11:47
Éditer	Copier	Supprimer	47	christelle79@wanadoo.fr	["ROLE_USER"]	$2y$13$vNw6ZAbJkKJN6.yMz1XSGux55LXGctg6gLhfk3C2Zqk...	Madeleine-Inès Rousseau	NULL	2023-06-04 19:11:47
Éditer	Copier	Supprimer	48	jerome.bertin@garcia.fr	["ROLE_USER"]	$2y$13$OST9oUzwmju5qrR.FuLiJ.d2CadyrkfGufcbcgX0Cj2...	Claudine de la Lagarde	NULL	2023-06-04 19:11:48
Éditer	Copier	Supprimer	49	david20@hotmail.fr	["ROLE_USER"]	$2y$13$kTUrwES6SfKyUdTaTLT5x.IXkWC/CMmPM3hco1vkxxs...	Gilles Coste	NULL	2023-06-04 19:11:48
Éditer	Copier	Supprimer	50	pgrenier@orange.fr	["ROLE_USER"]	$2y$13$nxwN8xhEdwCzXfKj9UQG5uyKKagOc/PJGmSEztvxKVZ...	Richard Coulon	NULL	2023-06-04 19:11:49
Éditer	Copier	Supprimer	51	albert@gmail.com	[]	$2y$13$t68ZbxqSudcTVDphnpS8b.pL3ZliryJQarRtatI1VNY...	albert	alberto	2023-06-06 08:40:24

Notre user s'est ajouté en bdd par contre son ROLE n'est pas renseigné

Dans notre controller nous allons définir le Rôle par défaut du user comme ceci

```
8   #[Route('/inscription', name: 'app_registration', methods:['GET','POST'
9   public function registration(Request $request, EntityManagerInterface $
0   {
1       $user = new User();
2       $user->setRoles(['ROLE_USER']);        You, 1 second ago • Uncommitt
3       $form = $this->createForm(RegistrationType::class,$user);
4
5       $form->handleRequest($request);
6       if($form->isSubmitted() && $form->isValid()){
7           $user=$form->getData();
```

Ajoutons donc elisabeth en pseudo nous aurons babet

Éditer	Copier	Supprimer	49	david20@hotmail.fr	["ROLE_USER"]	$2y$13$kTUrwES6SfKyUdTaTLT5x.lXkWC/CMmPM3hco1vkxxs...	Gilles Coste	NULL	2023-06-04 19:11:48
Éditer	Copier	Supprimer	50	pgrenier@orange.fr	["ROLE_USER"]	$2y$13$nxwN8xhEdwCzXfKj9UQG5uyKKagOc/PJGmSEztvxKVZ...	Richard Coulon	NULL	2023-06-04 19:11:49
Éditer	Copier	Supprimer	51	albert@gmail.com	[]	$2y$13$t68ZbxqSudcTVDphnpS8b.pL3ZliryJQarRtatI1VNY...	albert	alberto	2023-06-06 08:40:24
Éditer	Copier	Supprimer	53	babet@gmail.com	["ROLE_USER"]	$2y$13$zrAUNmt4UzsgAahH8PENkeCshBLoUaNGnYLzo4X2aEK...	elisabeth	babet	2023-06-06 08:51:20

Donc notre formulaire d'inscritpion fonctionne très bien, essayons par ailleurs de mettre des mots de passe différents

agrdr@grds.com

Mot de passe

- Les mots de passe ne correspondent pas

Confirmation du mot de passe

Nom

afza

Pseudo

afzd

Parfait cela fonctionne très bien au niveau du repeatedType

Le prochain chapitre est la modification du profil utilisateur